Kalman Filter State Estimation for Blue Jay Racing 23xt

Nikhil Mani

October 11, 2026

Contents

1 Introduction **1**

2 What is a Kalman Filter? **1**

3 Important vectors **5**
3.1 State Vector 5
3.2 Input Vector 5
3.3 Measurement Vector 6

4 Important Matrices **6**
4.1 F & G 6
4.2 H 7
4.3 Q and R 8

1 Introduction

This project is going to attempt to apply a Kalman filter-based approach to perform state estimation on the Blue Jay Racing 23xt. The end goal is for us to be able to produce 3D simulated replay of each race with accurate measurements of the position, velocity, and other dynamics at each point in time.

2 What is a Kalman Filter?

A Kalman filter is an "optimal filter," or a state estimation method that has formal guarantees on optimality in specific situations. The general idea of a Kalman filter is to condition your estimate of the system on your observations—and the variance associated with them-to get a better picture of the state. They do an extremely good job of sensor fusion for state estimation, so are used widely in autonomous navigation scenarios. Kalman filters also thrive in linear systems with Gaussian noise distributions.

On a high level, the Kalman filter works like this:

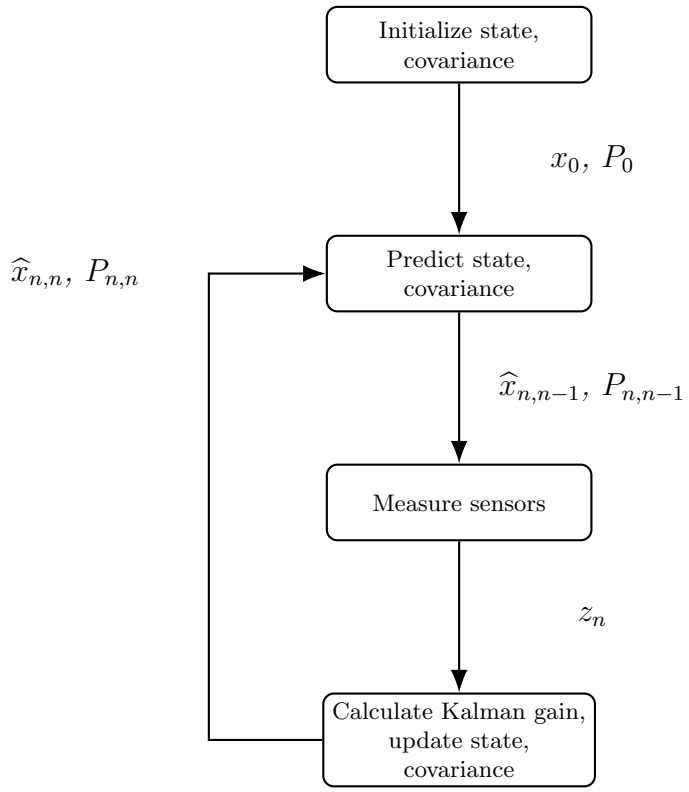


Figure 1: Kalman filter loop

Table 1: Summary of notation and key variables.

Symbol	**Definition**
$\widehat{x}_{a,b}$	Estimated state at time step a given information at time b
$P_{a,b}$	Estimated covariance matrix at time step a given information at time b
z_n	Measurment at time step n
K_n	Kalman Gain at time step n

Unfortunately, their strength in linear systems means they would struggle heavily when applied to a highly nonlinear system like a Baja track. Some extensions to the Kalman filter have been developed to better address nonlinear systems, like the Extended Kalman Filter and the Error-State Kalman Filter. These filters linearize dynamics around the current point by using Taylor series approximations. Our implementation will use an Error-State Kalman filter, as it aims to estimate the error state of the system (δx) instead of the state (x), leading to the linear approximations being more accurate. These filters work as outlined by the diagram below:

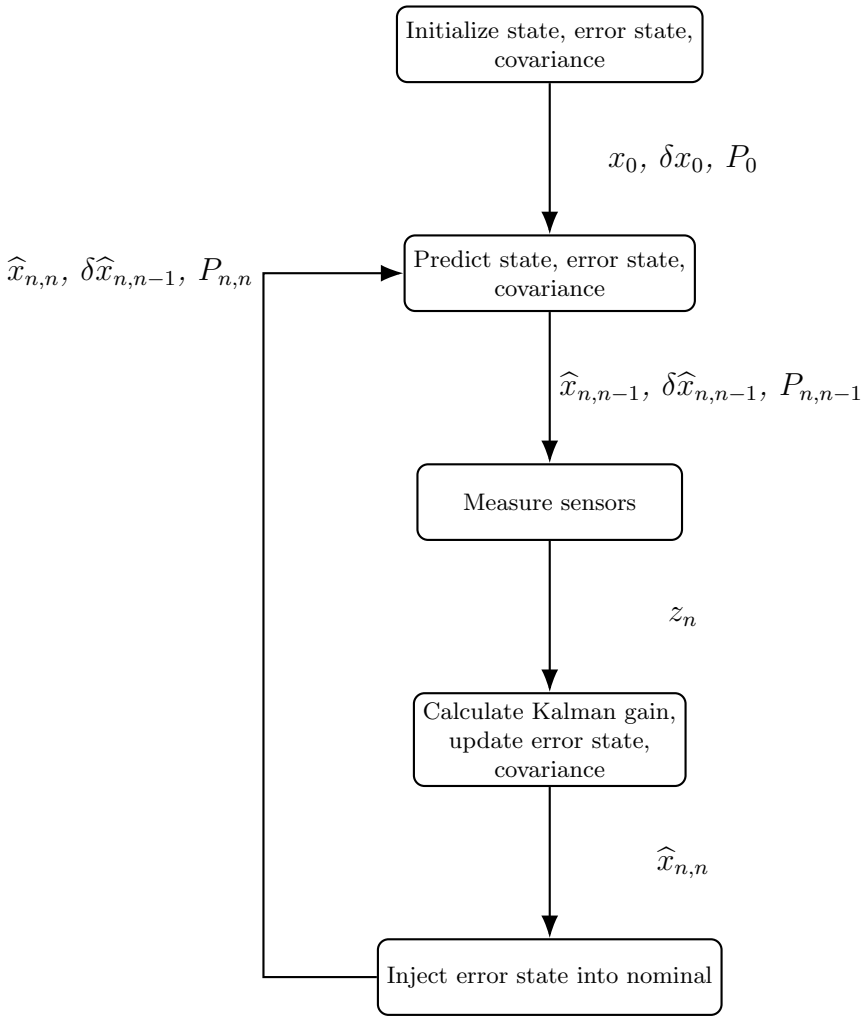


Figure 2: Kalman filter loop

3 Important vectors

3.1 State Vector

Our state vector must contain four main components:

- **Position**: Location in 3D space

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- **Velocity**

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}$$

- **Orientation**: Quaternion $q \in \mathbb{R}^4$

$$\hat{\boldsymbol{q}} = \begin{bmatrix} w \\ x \\ y \\ z \end{bmatrix}$$

- **Biases**: we must predict biases for both the accelerometer and the gyroscope:

$$\boldsymbol{b}_{\text{accel}} = \begin{bmatrix} b_{\text{ax}} \\ b_{\text{ay}} \\ b_{\text{az}} \end{bmatrix}$$

$$\boldsymbol{b}_{\text{gyro}} = \begin{bmatrix} b_{\text{gx}} \\ b_{\text{gy}} \\ b_{\text{gz}} \end{bmatrix}$$

Putting this all together, we end up with a 16×1 vector:

$$\boldsymbol{x}_{\text{state}} = \begin{bmatrix} \boldsymbol{x} \\ \dot{\boldsymbol{x}} \\ \hat{\boldsymbol{q}} \\ \boldsymbol{b}_{\text{accel}} \\ \boldsymbol{b}_{\text{gyro}} \end{bmatrix}$$

3.2 Input Vector

Our input vector is specified as below:

- **Acceleration**: Acceleration as measured by the IMU

$$\boldsymbol{a} = \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix}$$

- **Angular velocity:** Measured by the gyroscope in the IMU

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

So in total,

$$u = \begin{bmatrix} \boldsymbol{a} \\ \boldsymbol{\omega} \end{bmatrix}$$

3.3 Measurement Vector

Finally, we have our measurements; the values we check our proposed state against:

- **Steering Position**: Extrapolated steering heading for front wheels from measured steering encoder:

$$\boldsymbol{\delta} = \begin{bmatrix} \delta_{\text{in}} \\ \delta_{\text{out}} \end{bmatrix}$$

- **Wheel Encoders:** Measure the rotations per minute of each wheel:

$$\boldsymbol{n} = \begin{bmatrix} n_{\text{fl}} \\ n_{\text{fr}} \\ n_{\text{bl}} \\ n_{\text{br}} \end{bmatrix}$$

4 Important Matrices

4.1 F & G

The F Matrix is the state transition matrix and G is the input transition matrix. They are used as following:

$$x_{n+1,n} = Fx_{n,n} + Gu \tag{1}$$

F and G combine to give a prediction of the next state given the current state and inputs.

Our Error-State Kalman filter introduces 2 complications:

- We are actually focusing our tracking efforts on the error state, not the nominal state.
- We are assuming the system is nonlinear.

These complications lead to the following definitions:

$$\widehat{x}_{n+1,n} = f(\widehat{x}_{n,n}, u) \tag{2}$$

$$\widehat{\delta x}_{n+1,n} = (I + \frac{\partial f_{error}}{\partial \delta x}\Delta t)\widehat{\delta x}_{n,n} \tag{3}$$

The second line represents the derivative of a seperate f representing the nonlinear dynamics of the error state. Although these nonlinear dynamics can be hand-differentiated, we will use Python's autodiff as it will remain more accurate especially for later parts of the system and maintains flexibility in case we ever go down a learning-based path. Here is the explicit version of our F matrix for reference:

Our F is defined like this:

$$\begin{bmatrix} 0 & I & 0 & 0 & 0 \\ 0 & 0 & [\hat{R}\hat{a}]_x & -\hat{R} & 0 \\ 0 & 0 & -[\hat{w}]_x & 0 & -\hat{R} \\ & \dots & 0 & \dots & \\ & \dots & 0 & \dots & \end{bmatrix}$$

4.2 H

We formalize the concept of "conditioning our state on our observations" with the concept of Kalman gain, denoted as K_n. This quantity represents how much we trust new measurements, and how much we'll factor them into our updated state. Kalman gain for a linear Kalman filter is defined as:

$$K_n = P_{n,n-1}H^T \cdot (HP_{n,n-1}H^T + R_n)^{-1}$$

In scalar form, we can more intuitively write it as:

$$K_n = \frac{P_{n,n-1}}{P_{n,n-1} + R_n}$$

and we update our state with:

$$\widehat{x}_{n,n} = \widehat{x}_{n,n-1} + K_n(z_n - H\widehat{x}_{n,n-1})$$

Table 2: Matrix and variable notation.

Symbol	**Definition**
H	Maps state vector to expected measurement values
R	Sensor noise matrix

In scalar form, a Kalman gain of 1 correlates to fully trusting your measurement, and 0 vice versa. Let's look at the end behavior to get some intuition:

If $P \gg R$, $K \lim \to 1$. This physically means that your uncertainty on your state is extremely high, and so then regardless of how much noise our sensors have we trust the measurement 100%. This is commonly used near the beginning of the estimation process when we initialize a guess of the system.

If $R \gg P$, $K \lim \to 0$. This correlates with extremely high sensor noise, meaning we take our previous prediction with no weight on the new measurement. The more complex form does a similar process with some intermediary steps like projecting the covariance matrix into the measurement space, but the general idea remains the same.

The H matrix is much more granular than the F matrix, and operates directly on component vectors.

4.3 Q and R

These matrices represent the noise for different parts of our system.

- Q represents the noise of the measurements (process noise)
- R represents the noise of the sensors

I have not yet decided on the contents of either matrix. The R matrix is a lot easier to derive (we just measure noise), but the Q matrix is pretty tricky to figure out. Right now, I just have it set as a diagonal matrix with just one input affecting the state tying in. For example, position is set to 0 since nothing directly measures it, velocity has an acceleration term, etc. This requires some more work on my part to understand.